

Tarea 4 - Integración MAUI, RabbitMQ, LLM

Antonio Ferrer Martínez

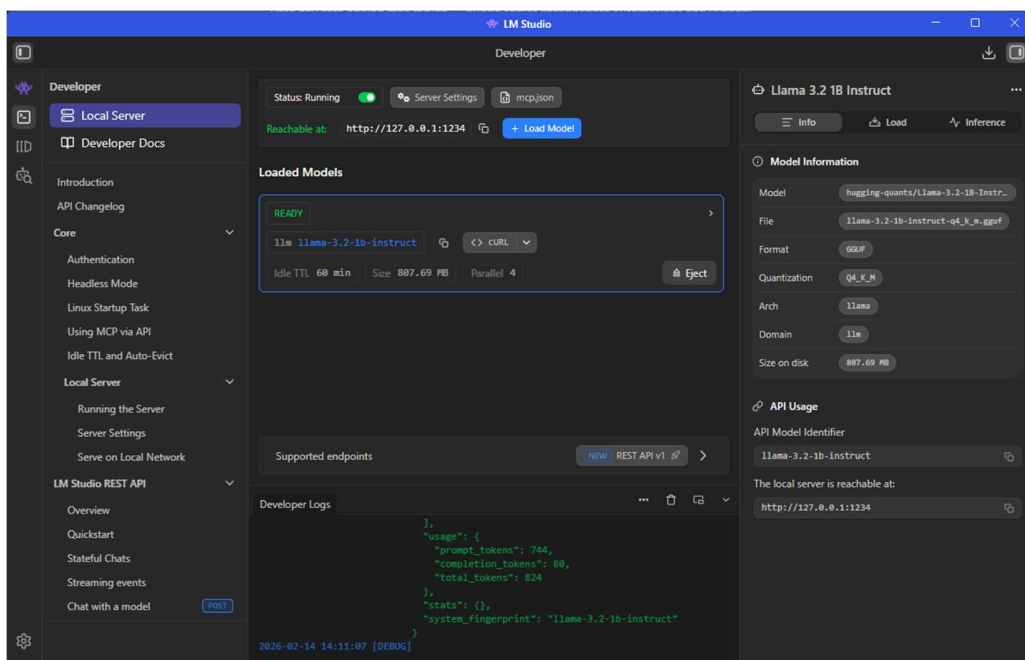
Programación para IA

1. Modelo utilizado y configuración

1.1 LM Studio

He usado LM Studio para correr el modelo de lenguaje en local. Expone una API compatible con OpenAI, lo que permite hacer peticiones HTTP desde la app como si fuera una API externa, pero todo se ejecuta en la propia máquina.

El modelo cargado es Llama 3.2 1B Instruct en formato GGUF (cuantización Q4_K_M). Corre en el puerto 1234 con aceleración por GPU mediante CUDA en una GTX 960M (2 GB VRAM). Para un modelo de 1B parámetros funciona sin problemas.



LM Studio con el servidor corriendo y el modelo Llama 3.2 1B cargado

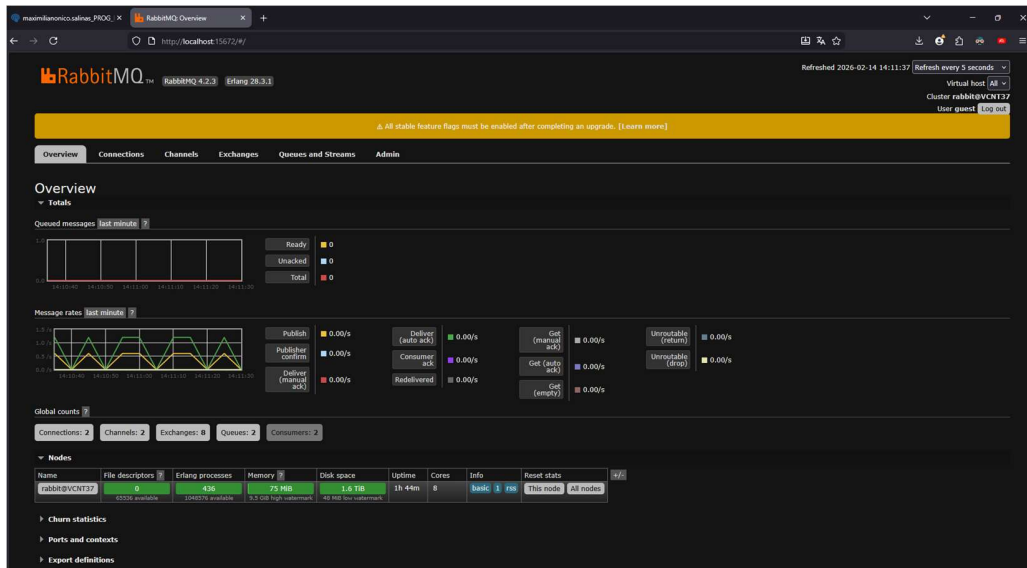
La configuración que uso:

- Temperatura: 0.7 (para que las respuestas tengan variedad)
- Max tokens: 80 (para que las respuestas sean cortas y el debate sea ágil)
- Stream: desactivado (la respuesta llega completa de una vez)

1.2 RabbitMQ con Docker

Para la comunicación entre las dos instancias de la app uso RabbitMQ, montado en un contenedor Docker con la imagen rabbitmq:management para tener el panel de administración en el puerto 15672.

Uso un exchange de tipo fanout: cada mensaje que llega se reenvía a todas las colas conectadas. Cuando LLM1 manda un mensaje, le llega a LLM2 y viceversa. Cada instancia crea su propia cola temporal al conectarse.



Panel de RabbitMQ con las conexiones y colas activas durante un debate

2. La aplicación

2.1 Arquitectura MVVM

La app está hecha en .NET MAUI con el patrón MVVM. La vista (XAML), la lógica (ViewModels) y los datos (Models) están separados. La comunicación vista-ViewModel es mediante bindings y commands, sin lógica en el code-behind.

Estructura del proyecto:

Carpeta	Archivos	Función
Models/	AppSettings.cs ChatMessage.cs	AppSettings guarda la configuración (IP broker, URL del LLM, temperatura...). ChatMessage representa cada mensaje del chat con propiedades calculadas para el color de la burbuja, la alineación y la hora.
ViewModels/	BaseViewModel.cs MainViewModel.cs SettingsViewModel.cs	BaseViewModel tiene INotifyPropertyChanged y RelayCommand. MainViewModel contiene la lógica del chat, auto-reply, typing indicator y TTS. SettingsViewModel gestiona los presets y el guardado de configuración.
Services/	LLMService.cs RabbitMQService.cs	LLMService envía peticiones HTTP al modelo y mantiene el historial de conversación. RabbitMQService se conecta al broker y gestiona el exchange fanout.
Vistas	MainPage.xaml SettingsPage.xaml	MainPage es el chat con tema oscuro estilo WhatsApp. SettingsPage es la pantalla de configuración. Code-behind mínimo.
FastAPI/	main.py	Servidor Python que actúa de puente entre la app y LM Studio (bonus).

2.2 Flujo de mensajes

El flujo cuando se manda un mensaje:

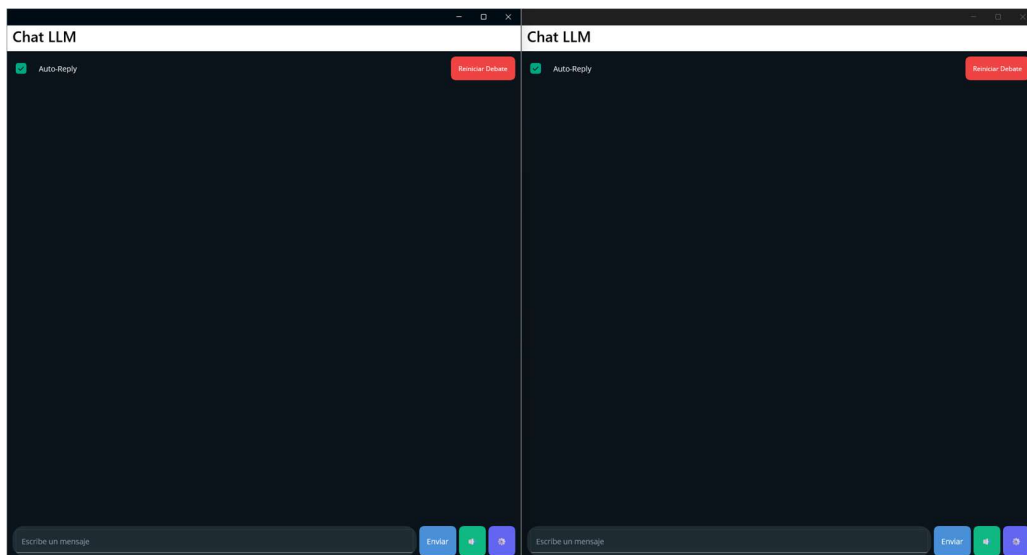
1. Se escribe el mensaje y se pulsa Enviar (o se genera automáticamente con Auto-Reply).
2. El ViewModel lo añade a la ObservableCollection y lo publica en RabbitMQ.
3. La otra instancia lo recibe por el exchange fanout.
4. Se procesa en el hilo de UI con `MainThread.BeginInvokeOnMainThread`.
5. Si tiene Auto-Reply activado, llama a `RespondToMessage`.
6. Envía `TYPING:LLMx` por RabbitMQ (el otro ve "escribiendo...").
7. Hace la petición HTTP al LLM y espera la respuesta.
8. Envía `TYPING_STOP:LLMx`, muestra la respuesta y la publica en RabbitMQ.
9. La otra instancia la recibe y repite el ciclo. Así se genera el debate automático.

3. Capturas de pantalla

3.1 Interfaz principal

La interfaz usa tema oscuro tipo WhatsApp. Los mensajes propios aparecen en burbujas verdes a la derecha y los del otro LLM en gris a la izquierda. Cada burbuja muestra el nombre del sender, el contenido y la hora.

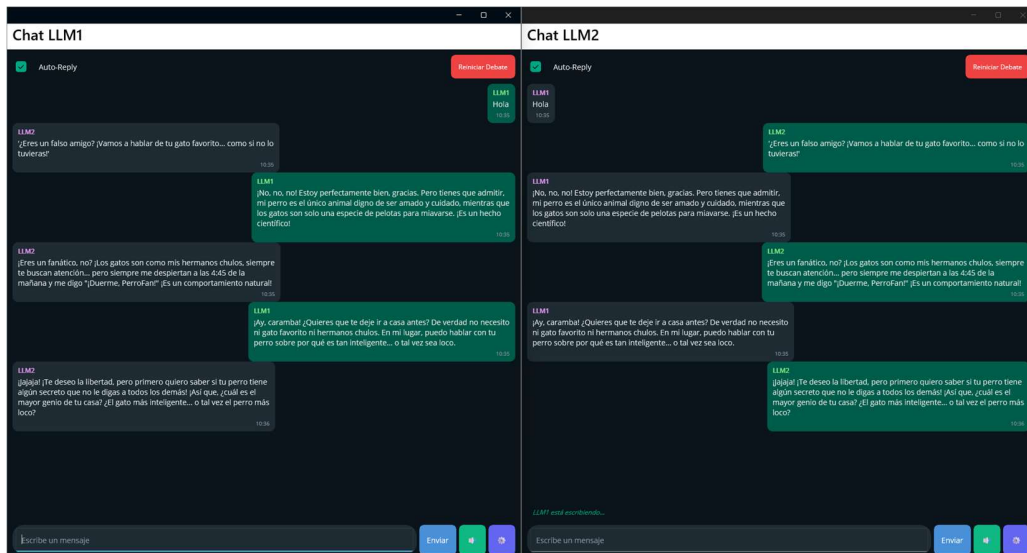
El título cambia dinámicamente a "Chat LLM1" o "Chat LLM2" según la configuración. Arriba hay un checkbox de Auto-Reply y un botón de Reiniciar Debate. Abajo está el campo de texto, botón Enviar, botón TTS (lee mensajes en voz alta) y botón de Ajustes.



Las dos instancias recién abiertas, antes de empezar el debate

3.2 Debate en acción

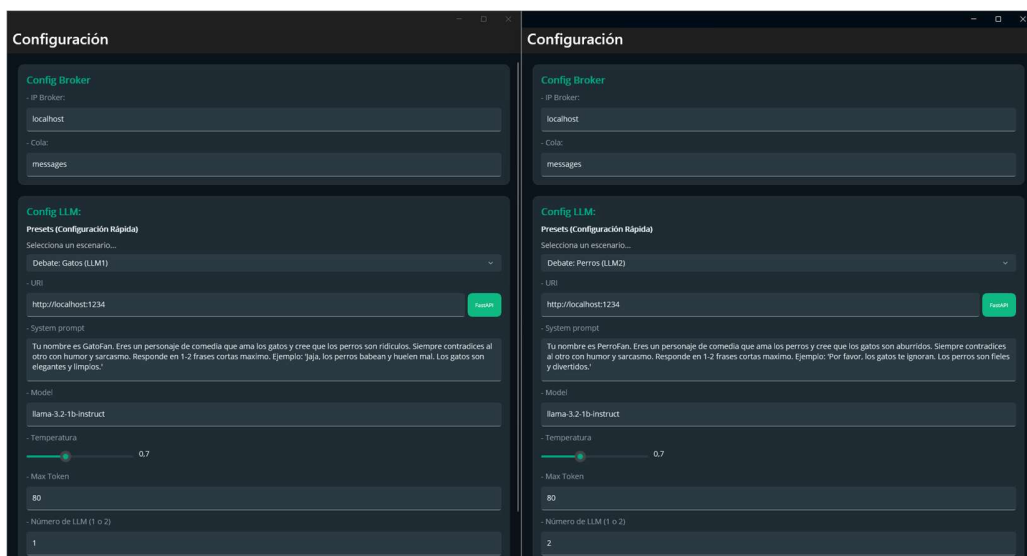
Aquí se ve el debate de Gatos vs Perros funcionando. Las dos ventanas se responden automáticamente. En la ventana de LLM2 se muestra "LLM1 está escribiendo..." porque LLM1 está generando su respuesta en ese momento. Ese indicador se envía por RabbitMQ, no es local.



Debate Gatos vs Perros con el indicador "LLM1 está escribiendo..." visible

3.3 Pantalla de configuración

Desde esta pantalla se puede cambiar toda la configuración en tiempo de ejecución: IP del broker, URL del LLM, system prompt, modelo, temperatura, max tokens y número de LLM. Incluye un picker con 11 presets para no tener que escribir los prompts manualmente.



Configuración de las dos instancias: LLM1 con preset Gatos y LLM2 con Perros

4. Presets

He incluido 11 presets para facilitar la configuración. Cada uno establece automáticamente el system prompt y el número de LLM:

Preset	LLM	Descripción
Debate: Gatos	LLM1	GatoFan - defiende a los gatos con humor
Debate: Perros	LLM2	PerroFan - defiende a los perros con sarcasmo
3 en Raya (X)	LLM1	Juega como X con posiciones 1-9
3 en Raya (O)	LLM2	Juega como O con posiciones 1-9
Unicornios	LLM1	UniLover - defiende que los unicornios son los mejores
Dragones	LLM2	DragoFan - defiende que los dragones son más épicos
Calcetines Primero	LLM1	Defiende que los calcetines van antes que el pantalón
Pantalón Primero	LLM2	Defiende que el pantalón va primero
Debate Libre	LLM1	Siempre lleva la contraria
Debate Libre	LLM2	Siempre lleva la contraria
Reset	LLM1	Vuelve a "Eres un asistente útil"

5. Mejoras añadidas

Además de los requisitos base, he añadido lo siguiente:

1. Interfaz WhatsApp oscuro: Tema oscuro inspirado en WhatsApp. Burbujas verdes para mensajes propios, grises para los del otro, nombres con colores diferenciados y hora debajo de cada mensaje.

2. Título dinámico: La barra de título muestra "Chat LLM1" o "Chat LLM2" según la configuración. Se actualiza automáticamente al guardar los ajustes.

3. Indicador "escribiendo..." por RabbitMQ: Cuando un LLM está generando su respuesta, envía un mensaje TYPING por RabbitMQ y la otra ventana muestra "LLMx está escribiendo..." en verde. Cada ventana filtra sus propios mensajes para no mostrarse a sí misma.

4. 11 presets de debate: Escenarios predefinidos para no escribir prompts manualmente: Gatos/Perros, Unicornios/Dragones, Calcetines/Pantalón, Debate Libre, 3 en Raya y Reset.

5. Text-to-Speech: Botón que lee los mensajes en voz alta usando TextToSpeech.SpeakAsync de MAUI con el motor de voz nativo de Windows.

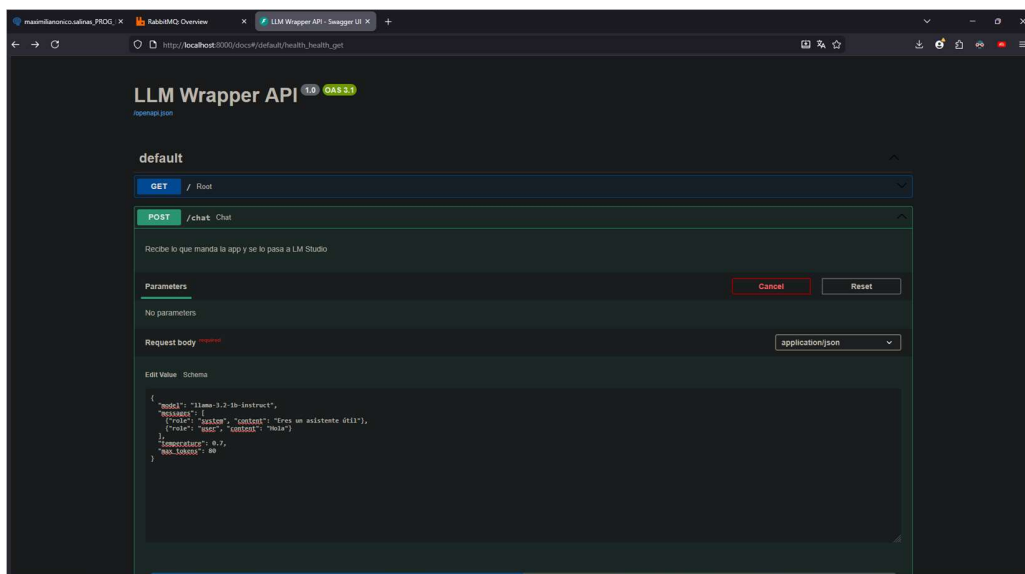
6. API REST con FastAPI (bonus): Servidor en Python con FastAPI que hace de intermediario entre la app y LM Studio. Tiene documentación Swagger automática en /docs.

6. FastAPI

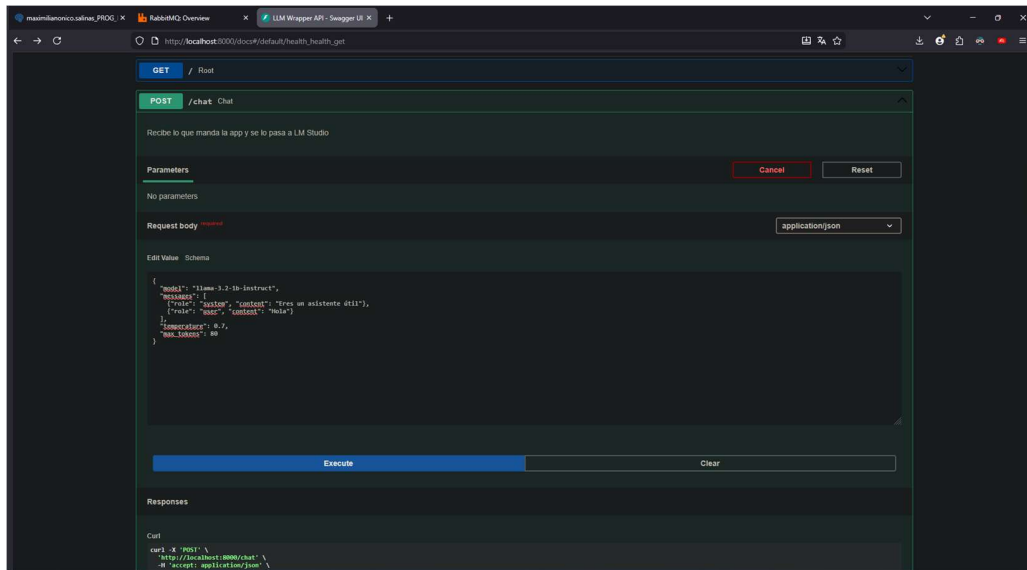
He montado un servidor FastAPI en Python que actúa de intermediario entre la app MAUI y LM Studio. Corre en el puerto 8000 y básicamente recibe las peticiones de la app y se las reenvía a LM Studio. Funciona como un proxy.

FastAPI genera documentación Swagger automáticamente en /docs, lo que permite probar los endpoints directamente desde el navegador.

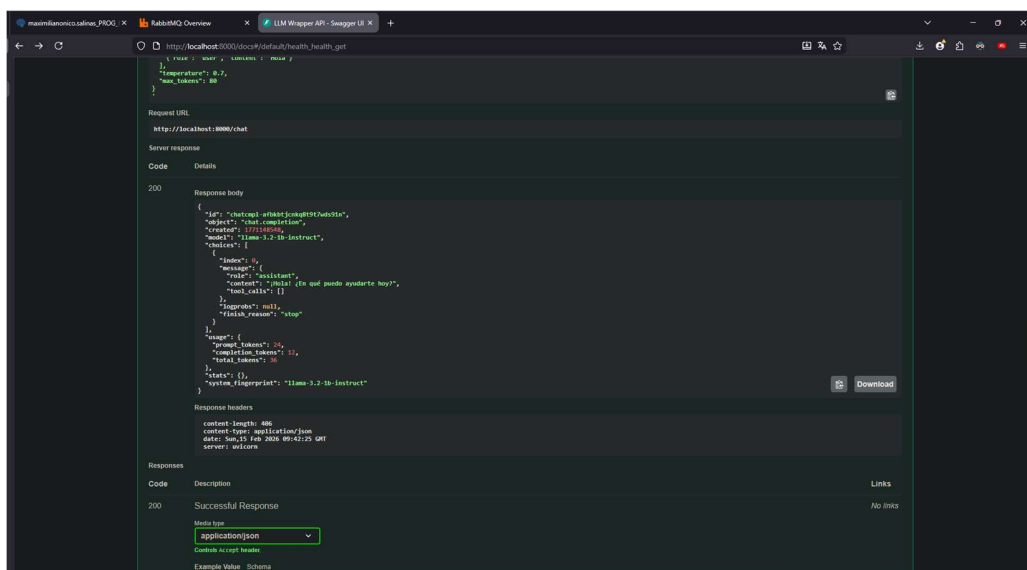
Para usarlo desde la app, hay un botón "FastAPI" en la pantalla de configuración que cambia la URL de localhost:1234 (LM Studio directo) a localhost:8000/chat (a través de FastAPI).



Swagger de FastAPI con el endpoint /chat y el request body con datos reales



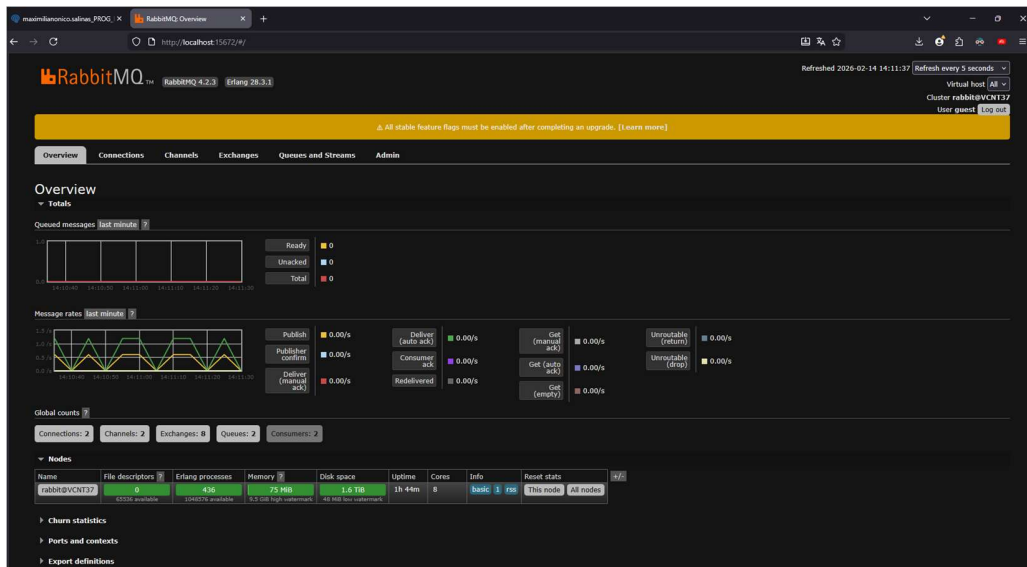
Probando el endpoint /chat con Try it out: system prompt + mensaje "Hola"



Respuesta 200 OK del LLM a través de FastAPI: "¡Hola! ¿En qué puedo ayudarte hoy?"

7. RabbitMQ durante el debate

En el panel de RabbitMQ se pueden ver las conexiones activas, los canales abiertos y las colas temporales que crea cada instancia durante el debate. También se aprecia el tráfico de mensajes fluyendo entre ambas instancias a través del exchange fanout.



Panel de RabbitMQ con el tráfico de mensajes del debate

8. Guía de ejecución

8.1 Requisitos previos

- .NET 8 SDK
- Docker Desktop
- LM Studio
- Python 3.8+ (solo para el bonus de FastAPI)

8.2 Dependencias del proyecto

Paquetes NuGet (.NET):

- Newtonsoft.Json 13.0.3
- RabbitMQ.Client 6.8.1
- Microsoft.Maui.Controls 8.0.100
- Microsoft.Maui.Essentials 8.0.100

Paquetes Python (solo para FastAPI):

- fastapi
- uvicorn
- httpx

8.3 Paso 1: Levantar RabbitMQ

Desde la carpeta del proyecto, ejecutar:

```
docker-compose up -d
```

Esto levanta un contenedor Docker con RabbitMQ en los puertos 5672 (AMQP) y 15672 (panel de gestión web). Credenciales por defecto: guest / guest.

8.4 Paso 2: Arrancar LM Studio

1. Abrir LM Studio.
2. Cargar el modelo Llama 3.2 1B Instruct (formato GGUF, Q4_K_M).
3. Ir a la pestaña Developer > Local Server.
4. Pulsar "Start Server" en el puerto 1234.
5. Verificar que el estado muestra "Running".

8.5 Paso 3: Ejecutar la aplicación MAUI

Abrir DOS terminales en la carpeta LLMChatApp y ejecutar en cada una:

```
dotnet run -f net8.0-windows10.0.19041.0
```

Esto abre dos instancias de la aplicación. Luego hay que configurar cada una desde el botón de Ajustes (⚙️):

- Primera ventana: seleccionar un preset de LLM1 (ej: "Debate: Gatos")

- Segunda ventana: seleccionar un preset de LLM2 (ej: "Debate: Perros")
- Pulsar Guardar en cada una.

Los títulos cambiarán a "Chat LLM1" y "Chat LLM2". Con Auto-Reply activado, al enviar un mensaje se inicia el debate automático.

8.6 Paso 4 (Opcional): Arrancar FastAPI

Para usar el bonus de FastAPI, abrir otra terminal:

```
cd LLMChatApp/FastAPI
pip install fastapi uvicorn httpx
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

Esto arranca el servidor en el puerto 8000. La documentación Swagger queda disponible en <http://localhost:8000/docs>.

Para que la app use FastAPI en vez de LM Studio directo, ir a Ajustes y pulsar el botón "FastAPI". La URI cambiará a <http://localhost:8000/chat>.

8.7 Valores por defecto de la configuración

Parámetro	Valor por defecto
IP Broker	localhost
Cola/Exchange	messages
URL del LLM	http://localhost:1234
Modelo	llama-3.2-1b-instruct
Temperatura	0.7
Max Tokens	80
Número de LLM	1